

Informe Ejercicio 5

Actualización de Tabla de Clasificación de Fútbol

El presente informe describe el desarrollo de un algoritmo en Python que permite actualizar la tabla de clasificación de un campeonato de fútbol después de jugarse una fecha. El programa registra los equipos, procesa los resultados de los partidos, calcula los puntos obtenidos y muestra la tabla ordenada de mayor a menor puntaje.

1. Descripción del Problema

Se requiere desarrollar un algoritmo que permita:

- i. Registrar la información inicial de los equipos.
- ii. Leer los resultados de los partidos jugados.
- iii. Actualizar partidos jugados, ganados, empatados y perdidos.
- iv. Actualizar goles a favor y goles en contra.
- v. Calcular los puntos obtenidos por cada equipo.
- vi. Mostrar la tabla de clasificación ordenada según la puntuación.

2. Desarrollo de la Solución

El problema se resolvió mediante funciones independientes que permiten organizar el código de forma clara. La solución incluye:

- i. **crear_equipo()**: crea el registro inicial de cada equipo.
- ii. **actualizar_partido()**: actualiza la información de los equipos después de cada partido.
- iii. **ordenar_tabla()**: ordena los equipos según los puntos obtenidos.
- iv. **mostrar_tabla()**: muestra la tabla final en pantalla.
- v. **main()**: coordina la ejecución del programa.

El algoritmo revisa el resultado de cada partido. Si un equipo gana, recibe tres puntos; si hay empate, ambos equipos reciben un punto; y si pierde, no suma puntos. Finalmente, la tabla se ordena de mayor a menor puntaje.

3. Código implementado

```
def crear_equipo(codigo):
    """
    Crea un registro inicial para un equipo.
    """
    return {
        "codigo": codigo,
        "jugados": 0,
        "ganados": 0,
        "empatados": 0,
        "perdidos": 0,
        "goles_favor": 0,
        "goles_contra": 0,
        "puntos": 0
    }

def actualizar_partido(
    equipos,
    codigo_local,
    goles_local,
```

```

codigo_visitante,
goles_visitante
):
    """
    Actualiza la información de los equipos después de un partido.
    """
    equipo_local = equipos[codigo_local]
    equipo_visitante = equipos[codigo_visitante]

    equipo_local["jugados"] += 1
    equipo_visitante["jugados"] += 1

    equipo_local["goles_favor"] += goles_local
    equipo_local["goles_contra"] += goles_visitante

    equipo_visitante["goles_favor"] += goles_visitante
    equipo_visitante["goles_contra"] += goles_local

    if goles_local > goles_visitante:
        equipo_local["ganados"] += 1
        equipo_visitante["perdidos"] += 1
        equipo_local["puntos"] += 3

    elif goles_local < goles_visitante:
        equipo_visitante["ganados"] += 1
        equipo_local["perdidos"] += 1
        equipo_visitante["puntos"] += 3

    else:
        equipo_local["empatados"] += 1
        equipo_visitante["empatados"] += 1
        equipo_local["puntos"] += 1
        equipo_visitante["puntos"] += 1

def ordenar_tabla(equipos):
    """
    Ordena la tabla de clasificación por puntos de mayor a menor.
    """
    return sorted(
        equipos.values(),
        key=lambda equipo: equipo["puntos"],
        reverse=True
    )

def mostrar_tabla(tabla):
    """
    Muestra la tabla de clasificación.
    """
    print("\nTabla de clasificación")
    print("Código | PJ | PG | PE | PP | GF | GC | PTS")

    for equipo in tabla:
        print(
            equipo["codigo"],
            "|",
            equipo["jugados"],
            "|",
            equipo["ganados"],
            "|",
            equipo["empatados"],
            "|",
            equipo["perdidos"],
            "|",
            equipo["goles_favor"],
            "|",
            equipo["goles_contra"],
            "|",
            equipo["puntos"]

```

```

    )

def main():
    """
    Ejecuta el programa principal.
    """
    equipos = {}

    cantidad_equipos = int(input("Ingrese la cantidad de equipos: "))

    for i in range(cantidad_equipos):
        codigo = input(f"Ingrese el código del equipo {i + 1}: ")
        equipos[codigo] = crear_equipo(codigo)

    cantidad_partidos = int(input("\nIngrese la cantidad de partidos: "))

    for i in range(cantidad_partidos):
        print(f"\nPartido {i + 1}")

        codigo_local = input("Código equipo local: ")
        goles_local = int(input("Goles equipo local: "))

        codigo_visitante = input("Código equipo visitante: ")
        goles_visitante = int(input("Goles equipo visitante: "))

        actualizar_partido(
            equipos,
            codigo_local,
            goles_local,
            codigo_visitante,
            goles_visitante
        )

    tabla_ordenada = ordenar_tabla(equipos)
    mostrar_tabla(tabla_ordenada)

if __name__ == "__main__":
    main()

```

4. Prueba de Escritorio

Se utilizaron tres equipos: A, B y C.

Partido	Local	Marcador	Visitante
1	A	2 - 1	B
2	A	0 - 0	C
3	B	3 - 2	C

Resultado final:

Equipo	PJ	PG	PE	PP	GF	GC	PTS
A	2	1	1	0	2	1	4
B	2	1	0	1	4	4	3
C	2	0	1	1	2	3	1

5. Pruebas unitarias

Se realizaron pruebas unitarias para comprobar:

- i. Victoria del equipo local.
- ii. Victoria del equipo visitante.
- iii. Partido empatado.
- iv. Orden correcto de la tabla.

Al ejecutar las pruebas unitarias, todas las validaciones fueron exitosas.

Código

```
import unittest

from ejercicio5_futbol import (
    crear_equipo,
    actualizar_partido,
    ordenar_tabla
)

class TestFutbol(unittest.TestCase):
    """
    Pruebas unitarias para validar la tabla de clasificación.
    """

    def test_equipo_local_gana(self):
        equipos = {
            "A": crear_equipo("A"),
            "B": crear_equipo("B")
        }

        actualizar_partido(equipos, "A", 2, "B", 1)

        self.assertEqual(equipos["A"]["puntos"], 3)
        self.assertEqual(equipos["A"]["ganados"], 1)
        self.assertEqual(equipos["B"]["perdidos"], 1)

    def test_equipo_visitante_gana(self):
        equipos = {
            "A": crear_equipo("A"),
            "B": crear_equipo("B")
        }

        actualizar_partido(equipos, "A", 0, "B", 2)

        self.assertEqual(equipos["B"]["puntos"], 3)
        self.assertEqual(equipos["B"]["ganados"], 1)
        self.assertEqual(equipos["A"]["perdidos"], 1)

    def test_partido_empatado(self):
        equipos = {
            "A": crear_equipo("A"),
            "B": crear_equipo("B")
        }

        actualizar_partido(equipos, "A", 1, "B", 1)

        self.assertEqual(equipos["A"]["puntos"], 1)
        self.assertEqual(equipos["B"]["puntos"], 1)
        self.assertEqual(equipos["A"]["empatados"], 1)
        self.assertEqual(equipos["B"]["empatados"], 1)

    def test_ordenar_tabla(self):
        equipos = {
            "A": crear_equipo("A"),
            "B": crear_equipo("B")
        }

        actualizar_partido(equipos, "A", 3, "B", 1)
```

```
tabla = ordenar_tabla(equipos)

self.assertEqual(tabla[0]["codigo"], "A")

if __name__ == "__main__":
    unittest.main()
```

6. Conclusión

El desarrollo de este ejercicio permitió entender cómo actualizar una tabla de posiciones a partir de los resultados de los partidos. También ayudó a practicar el manejo de datos organizados por equipo, el cálculo de puntos y el ordenamiento de información. Las pruebas realizadas permitieron comprobar que el programa funciona correctamente en diferentes situaciones.